



Regular Expressions



Regular Expressions

Regular Expressions are a powerful tool for various kinds of string manipulation.

These are basically a special text string that is used for describing a search pattern to extract information from text such as code, files, log, spreadsheets, or even documents.

Regular Expressions

Regular expressions are a *domain specific language (DSL)* that is present as a library in most of the modern programming languages, besides Python.

A *regular expression* is a special sequence of characters that helps to match or find strings in another string.

In Python, regular expressions can be accessed using the `re` module which comes as a part of the **Standard Library**

Regular Expressions

The Match Function

As the name suggest, the `match()` function matches a pattern to string with optional flags.

The syntax of `match()` function is,

`re.match (pattern, string, flags=0)`

Flag	Description
re.I	Case sensitive matching
re.M	Matches at the end of the line
re.X	Ignores whitespace characters
re.U	Interprets letters according to Unicode character set

EXAMPLE- Match() Function

Examples:

```
import re
string = "She sells sea shells on the sea shore"
pattern1 = "sells"
if re.match(pattern1, string):
    print("Match Found")
else:
    print(pattern1, "is not present in the string")
pattern2 = "She"
if re.match(pattern2, string):
    print("Match Found")
else:
    print(pattern2, "is not present in the string")
```

OUTPUT

```
sells is not present in the string
Match Found
```

The re.match() function finds match at the beginning.

The search Function

The **search()** function in the re module searches for a pattern anywhere in the string. Its syntax can be given as, **re.search(pattern, string, flags=0)**

The syntax is similar to the match() function. The function searches for first occurrence of *pattern* within a *string* with optional *flags*. If the search is successful, a *match* object is returned and **None** otherwise.

Example:

```
import re
string = "She sells sea shells on the sea shore"
pattern = "sells"
if re.search(pattern, string):
    print("Match Found")
else:
    print(pattern, "is not present in the string")
```

OUTPUT

Match Found

The sub() Function

The **sub()** function in the re module can be used to search a pattern in the string and replace it with another pattern. The syntax of sub() function can be given as, **re.sub(pattern, repl, string, max=0)**

According to the syntax, the sub() function replaces all occurrences of the pattern in string with repl, substituting all occurrences unless any max value is provided. This method returns modified string.

Example:

```
import re
string = "She sells sea shells on the sea shore"
pattern = "sea"
repl = "ocean"
new_string = re.sub(pattern, repl, string, 1)
print(new_string)
```

OUTPUT

```
She sells ocean shells on the sea shore
```

The findall() Function

The **findall()** function is used to search a string and returns a list of matches of the pattern in the string. If no match is found, then the returned list is empty. The syntax of match() function can be given as,

matchList = re.findall(pattern, input_str, flags=0)

Examples:

```
import re
string = "She sells sea shells on the sea shore"
pattern = "se"
matches = re.findall(pattern, string)
for item in matches:
    print(item, end=' ')
```


Flag Options

The `search()`, `findall()` and `match()` functions of the module take options to modify the behavior of the pattern match. Some of these flags are:

re.I or re.IGNORECASE — Ignores case of characters, so "Match", "MATCH", "mAtCh", etc are all same

re.S or re.DOTALL — Enables dot (.) to match newline. By default, dot matches any character other than the newline character.

re.M or re.MULTILINE — Makes the ^ and \$ to match the start and end of each line. That is, it matches even after and before line breaks in the string. By default, ^ and \$ matches the start and end of the whole string.

re.L or re.LOCALE- Makes the flag \w to match all characters that are considered letters in the given current locale settings.

re.U or re.UNICODE- Treats all letters from all scripts as word characters.

Metacharacters in Regular Expression

Metacharacter	Description	Example	Remarks
<code>^</code>	Matches at the beginning of the line.	<code>^Hi</code>	It will match Hi at the start of the string.
<code>\$</code>	Matches at the end of the line.	<code>Hi\$</code>	It will match Hi at the end of the string.
<code>.</code>	Matches any single character except the newline character.	<code>Lo.</code>	It will match Lot, Log, etc.
<code>[...]</code>	Matches any single character in brackets.	<code>[Hh]ello</code>	It will match "Hello" or "hello".
<code>[^...]</code>	Matches any single character not in brackets.	<code>[^aeiou]</code>	It will match anything other than a lowercase vowel.
<code>re*</code>	Matches 0 or more occurrences of regular expression.	<code>[a-z]*</code>	It will match zero or more occurrence of lowercase characters.
<code>re+</code>	Matches 1 or more occurrence of regular expression.	<code>[a-z]+</code>	It will match one or more occurrence of lowercase characters
<code>re?</code>	Matches 0 or 1 occurrence of regular expression.	<code>Book?</code>	It will match "Book" or "Books".
<code>re{n}</code>	Matches exactly n number of occurrences of regular expression.	<code>42{1}5</code>	It will match 425.
<code>re{n,}</code>	Matches n or more occurrences of regular expression.	<code>42{1,}5</code>	It will match 42225 or any number with more than one 2s between 4 and 5.
<code>re{n,m}</code>	Matches at least n and at most m occurrences of regular expression.	<code>42{1,3}5</code>	It will match 425, 4225, 42225.

Metacharacters in Regular Expression

<code>a b</code>	Matches either a or b.	<code>"Hello" "Hi"</code>	It will match Hello or Hi.
<code>\w</code>	Matches word characters.	<code>re.search(r'\w', 'xx123xx')</code>	Match will be made.
<code>\W</code>	Matches non-word characters.	<code>if(re.search(r'\W', '@#\$%')): print("Done")</code>	Done
<code>\s</code>	Matches whitespace, equivalent to <code>[\t\n\r\f]</code> .	<code>if(re.search(r'\s',"abcdsd")): print("Done")</code>	Done
<code>\S</code>	Matches non-whitespace, equivalent to <code>^[^t\n\r\f]</code> .	<code>if(re.search(r'\S'," abcdsd")): print("Done")</code>	Done
<code>\d{n}</code>	Matches exactly n digits.	<code>\d{2}</code>	It will match exactly 2 digits.
<code>\d{n,}</code>	Matches n or more digits.	<code>\d{3,}</code>	It will match 3 or more digits.
<code>\d{n,m}</code>	Matches n and at most m digits.	<code>\d{2,4}</code>	It will match 2,3 or 4 digits.
<code>\D</code>	Matches non-digits.	<code>(\D+\d)</code>	It will match Hello 5678, or any string starting with no digit followed by digits(s).
<code>\A</code>	Matches beginning of the string.	<code>\AHi</code>	It will match Hi at the beginning of the string.
<code>\Z</code>	Matches end of the string.	<code>Hi\Z</code>	It will match Hi at the end of the string.
<code>\G</code>	Matches point where last match finished.	<code>import re if(re.search(r'\Gabc','abcba cab')): print("Done") else: print("Not Done")</code>	Not Done
<code>\b</code>	Matches word boundaries when outside brackets. Matches backspace when inside brackets.	<code>\bHi\b</code>	It will match Hi at the word boundary.
<code>\B</code>	Matches non-word boundaries.	<code>\bHi\B</code>	Hi should start at word boundary but end at a non-boundary as in High
<code>\n, \t, etc.</code>	Matches newlines, tabs, etc.	<code>re.search(r'\t', '123 \t abc ')</code>	Match will be made.

Character Classes

When we put the characters to be matched inside square brackets, we call it a **character class**. For example, `[aeiou]` defines a character class that has a vowel character.

Examples:

```
import re
pattern=r"[aeiou]"
if re.search(pattern,"clue"):
    print("Match clue")
if re.search(pattern,"bcdfg"):
    print("Match bcdfg")
```

OUTPUT

```
Match clue
```

Groups

A **group** is created by surrounding a part of the regular expression with parentheses. You can even give group as an argument to the metacharacters such as * and ?.

Example:

```
import re
pattern = r"gr(ea)*t"    # group of ea created
if re.match(pattern, "great"):
    print("Match ea")
if re.match(pattern, "greaeaeaeaeaeaeat"):
    print("Match greaeaeaeaeaeaeat")
```

OUTPUT

```
Match ea
Match greaeaeaeaeaeaeat
```

The content of groups in a match can be accessed by using the `group()` function. For example,

- `group(0)` or `group()` returns the whole match.
- `group(n)`, where `n` is greater than 0, returns the `n`th group from the left.
- `group()` returns all groups up from 1.